

TypeScript, NPM, and Vite

Part 1

TypeScript Fundamentals & Package Management

Chapter Overview

- 01** TypeScript Foundations
- 02** Type System Deep Dive
- 03** Functions and Generics
- 04** NPM and Node Modules



TypeScript Foundations

Section 1



Why TypeScript?

The Problem with JavaScript

- No built-in static type system — errors are typically caught at runtime
- No editor intelligence — autocomplete guesses instead of knows

What TypeScript Adds

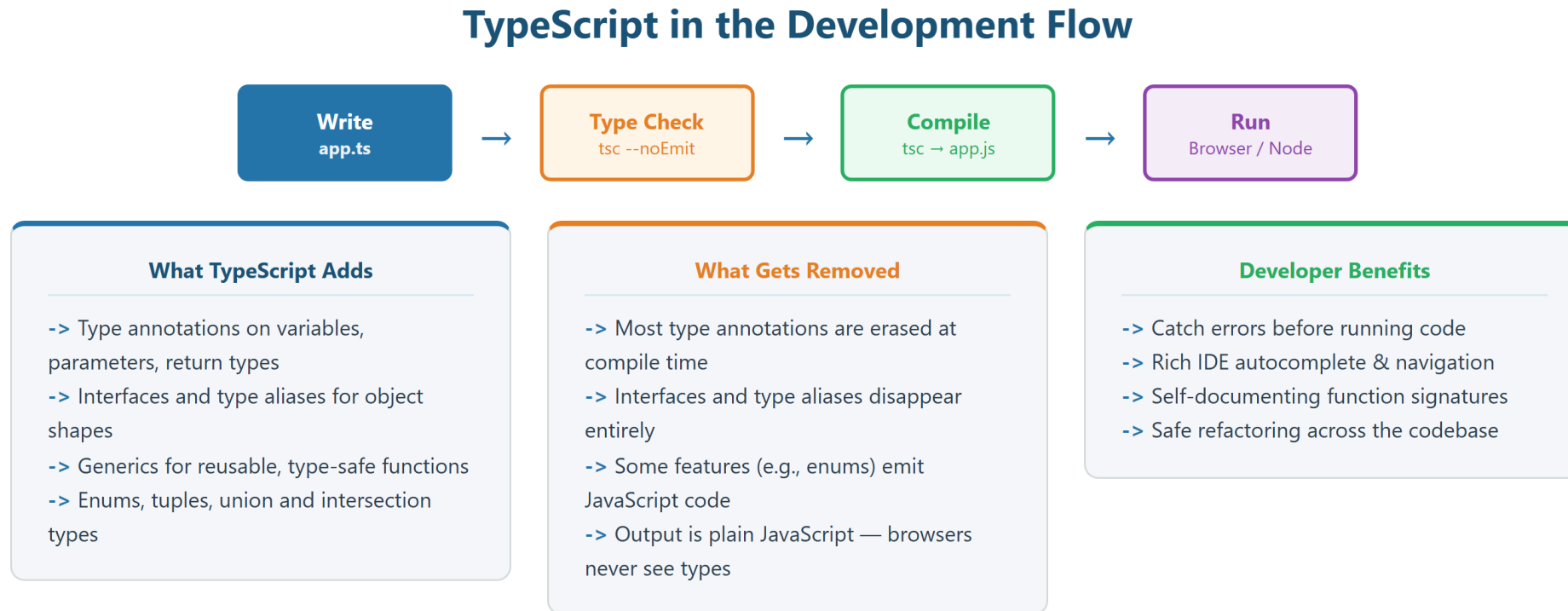
- Static type checking — catch errors before code runs
- Rich editor support — autocomplete, refactoring, inline docs

TypeScript Is a Superset

- All valid JavaScript is valid TypeScript — you adopt types incrementally
- TypeScript compiles to plain JavaScript — browsers never see the types



TypeScript in the Development Flow



Setting Up TypeScript

Installing TypeScript

- `npm install -D typescript` — install as a dev dependency
- `npx tsc --init` — generates `tsconfig.json`

Key `tsconfig.json` Options

- `target: ES2022` — which JS version to compile to
- `strict: true` — enable all strict type-checking options
- `module: ESNext` — use ES modules
- `outDir: ./dist` — where compiled JS goes



tsconfig.json Essentials

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "strict": true,
    "esModuleInterop": true,
    "jsx": "react-jsx",           // modern JSX transform (React projects)
    "outDir": "./dist",
    "rootDir": "./src",
    "moduleResolution": "bundler",
    "skipLibCheck": true
  },
  "include": ["src/**/*"]
}
```



Basic Type Annotations

Primitive Types

- string, number, boolean — the three you use most
- null, undefined — explicit absence of value

Type Inference

- TypeScript infers types from assigned values — `let count = 0` is already number
- Annotate when the type isn't obvious from the value

When to Annotate vs Infer

- Function parameters: always annotate
- Variable declarations: let TypeScript infer when possible



Type Annotations in Practice

```
// Explicit annotations
let name: string = "Alice";
let age: number = 30;
let active: boolean = true;

// Type inference – no annotation needed
let count = 0;           // TypeScript knows: number
let greeting = "Hello"; // TypeScript knows: string

// Arrays
let scores: number[] = [95, 87, 92];
let names: string[] = ["Alice", "Bob"];

// Union types – more than one possibility
let id: string | number = "abc-123";
id = 42; // also valid
```

Type System Deep Dive

Section 2



Interfaces

Defining Object Shapes

- An interface describes what properties an object must have
- Optional properties use ? — not every object needs every field
- Readonly properties prevent mutation after creation

Why Interfaces Matter

- Contracts between parts of your code — if it doesn't match, TypeScript tells you
- React props are typed with interfaces — you'll use this pattern every day



Interfaces in Practice

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  role?: string;           // optional  
  readonly createdAt: Date; // immutable  
}  
  
const alice: User = {  
  id: 1,  
  name: "Alice",  
  email: "alice@example.com",  
  createdAt: new Date(),  
};  
  
// alice.createdAt = new Date(); // Error: readonly
```

Type Aliases vs Interfaces

Type Aliases

- `type Status = 'active' | 'inactive' | 'pending'` — union types
- `type Pair = [string, number]` — tuple types
- Can describe any type shape, not just objects

Interfaces

- Best for object shapes — especially React props and API responses
- Support extends for inheritance

Rule of Thumb

- Use interface for objects, type for everything else



Type Aliases vs Interfaces

Type Aliases vs Interfaces

interface

```
interface User {  
  name: string;  
  age: number;  
}  
  
// Extend with inheritance  
interface Admin extends User {  
  role: string;  
}
```

- > ✓ Best for object shapes
- > ✓ Supports **extends**
- > ✓ Common for React props and API object shapes
- > ✓ Can be merged (declaration merging)

vs

type

```
type Status = "active" | "inactive";  
  
type Pair = [string, number];  
  
// Combine with intersection  
type Admin = User & {  
  role: string;  
};
```

- > ✓ Union types (A | B)
- > ✓ Tuple types ([string, number])
- > ✓ Intersection types (A & B)
- > ✓ Mapped / conditional types

Rule of thumb: **interface** for objects · **type** for everything else



Extending Interfaces

```
interface Animal {  
  name: string;  
  sound: string;  
}  
  
interface Dog extends Animal {  
  breed: string;  
}  
  
const rex: Dog = {  
  name: "Rex",  
  sound: "Woof",  
  breed: "German Shepherd"  
};  
  
// Intersection types (type alias approach)  
type Employee = User & { department: string };
```



Literal and Union Types

Literal Types

- Narrow a type to exact values: 'success' | 'error' | 'loading'
- Safer than plain string — the compiler catches typos

Discriminated Unions

- Add a shared literal property to distinguish union members
- TypeScript narrows the type in switch/if blocks automatically
- This pattern is used heavily in React state management



Union Types and Type Narrowing

```
type Status = 'loading' | 'success' | 'error';

interface LoadingState { status: "loading" }
interface SuccessState { status: "success"; data: string[] }
interface ErrorState   { status: "error"; message: string }

type AppState = LoadingState | SuccessState | ErrorState;

function render(state: AppState): string {
  switch (state.status) {
    case "loading": return "Loading...";
    case "success": return state.data.join(", ");
    case "error":   return `Error: ${state.message}`;
  }
}
```

Functions and Generics

Section 3



Typing Functions

Parameter Annotations

- Function parameters should be explicitly typed — TypeScript can infer in some cases, but explicit types are best practice

Return Type Annotation

- TypeScript can infer return types, but explicit annotations catch mistakes

Function Types

- `(a: number, b: number) => number` — type for a variable holding a function
- Used for callbacks, event handlers, and React props

Optional and Default Parameters

- Optional: `(name?: string)` — may be undefined
- Default: `(name = 'World')` — has a fallback value



Typed Functions

```
// Explicit parameter and return types
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function with type annotation
const greet = (name: string): string => `Hello, ${name}`;

// Function type as a variable
type MathOp = (a: number, b: number) => number;
const multiply: MathOp = (a, b) => a * b;

// Optional and default parameters
function log(msg: string, level: string = "info"): void {
  console.log(`[${level}] ${msg}`);
}
```

Generics

Why Generics?

- Write one function that works with any type, without losing type safety
- Without generics, you'd use 'any' — which disables all checking

Generic Syntax

- `<T>` is a type parameter — a placeholder filled in when the function is called
- Convention: T for type, K for key, V for value

Where You'll See Generics

- React: `useState<number>(0)`, `useRef<HTMLDivElement>(null)`
- API calls: `fetch<User[]>('/api/users')`
- Array methods: `Array<string>`, `Map<string, User>`



Generics in Practice

```
// Generic function
function first<T>(items: T[]): T | undefined {
  return items[0];
}

first<number>([1, 2, 3]); // returns number
first<string>(["a", "b"]); // returns string
first([true, false]); // inferred: boolean

// Generic interface
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

type UserResponse = ApiResponse<User>;
```

Generic Constraints

```
// Constrain T to objects with a "length" property
function logLength<T extends { length: number }>(item: T): void {
    console.log(item.length);
}

logLength("hello");      // OK: string has length
logLength([1, 2, 3]);    // OK: array has length
// logLength(42);        // Error: number has no length

// keyof constraint – K must be a key of T
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const user = { name: "Alice", age: 30 };
getProperty(user, "name"); // "Alice" – type: string
```

NPM and Node Modules

Section 4



What Is NPM?

Node Package Manager

- One of the world's largest software registries — millions of open-source packages
- Installed automatically with Node.js

Three Roles

- Registry: a database of open-source packages
- CLI: the npm command to install, update, and manage packages
- Website: npmjs.com — search, documentation, and security advisories



NPM Ecosystem

NPM Ecosystem

npm Registry

Millions of open-source packages

react typescript vite eslint express

npm CLI

```
npm init -y
npm install react
npm install -D typescript
npm run build
npm run dev
```

package.json

- > name & version
- > scripts (dev, build, test)
- > dependencies
- > devDependencies

dependencies vs devDependencies

dependencies

- > Needed at runtime
- > react, react-dom, axios
- > Shipped to production

devDependencies

- > Needed only for development
- > typescript, eslint, vite
- > NOT shipped to production

node_modules & package-lock.json

- > **node_modules/** — installed packages (never commit)
- > **package-lock.json** — exact versions for reproducible installs
- > Use **npm ci** in CI for clean, reproducible installs from package-lock.json

package.json — Your Project Manifest

What It Contains

- name, version — project identity
- scripts — custom commands (dev, build, test, lint)
- dependencies — packages your app needs to run
- devDependencies — packages needed only during development

Key NPM Commands

- npm init -y — create a new package.json with defaults
- npm install <pkg> — add a dependency
- npm install -D <pkg> — add a dev dependency
- npm run <script> — execute a script from package.json



package.json Example

```
{
  "name": "my-ts-app",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "lint": "eslint src/"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "typescript": "^5.6.0",
    "vite": "^6.0.0",
    "@types/react": "^18.3.0"
  }
}
```

ES Modules in TypeScript

Import and Export

- Named exports: `export function add() / import { add } from './math'`
- Default exports: `export default class App / import App from './App'`
- Re-exports: `export { add } from './math'` — barrel files

Type-Only Imports

- `import type { User } from './types'` — stripped at compile time
- Makes it clear the import is only used for type checking

Module Resolution

- TypeScript resolves imports based on `moduleResolution` in `tsconfig`
- Use 'bundler' for Vite projects — matches how Vite resolves modules



Modules in TypeScript

```
// math.ts – named exports
export function add(a: number, b: number): number {
  return a + b;
}
export const PI = 3.14159;

// types.ts – type-only exports
export interface User {
  id: number;
  name: string;
}

// app.ts – imports
import { add, PI } from "./math";
import type { User } from "./types";

const result = add(2, 3);
```

Type Declarations for JavaScript Libraries

The @types Problem

- Most npm packages are written in JavaScript — no type information
- DefinitelyTyped provides type declarations for thousands of packages
- Install with `npm install -D @types/<package>`

Examples

- `@types/react`, `@types/node`, `@types/express`
- Some packages include their own types (no @types needed)



Key Takeaways — Part 1

Key Takeaways

1. TypeScript adds static type checking to JavaScript — catch bugs at compile time, not runtime
2. Use interfaces for object shapes (especially React props) and type aliases for unions and tuples
3. Generics let you write reusable, type-safe code — you'll use them with React hooks and API calls
4. NPM manages dependencies; package.json is your project manifest with scripts, deps, and metadata
5. ES Modules (import/export) organize code into files — use type-only imports for interfaces
6. Next: Part 2 covers Vite build tooling, project architecture, and preparing for React

